

Q1. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

I would choose the **ESP32** because it has built-in Wi-Fi and Bluetooth, which makes it easy to send sensor data to a mobile application every 5 seconds. It is also much faster than the Arduino UNO and has more memory, so it can handle multiple tasks like reading the soil moisture sensor, controlling the water pump, and communicating with the mobile app at the same time.

If internet connectivity was not required, then I would choose the **Arduino UNO**. It is cheaper, consumes less power for simple applications, and is sufficient for monitoring the soil moisture sensor and controlling the water pump locally.

Q2. Compare Wi-Fi, Bluetooth, LoRa, and MQTT. Which would you choose?

- **Wi-Fi** provides high-speed communication but consumes more power. It is suitable when internet access is available.
- **Bluetooth** uses less power but has a short communication range. It is mainly used for nearby devices.
- **LoRa** has a very long communication range of several kilometers and consumes very little power, but its data speed is low.
- **MQTT** is not a wireless technology. It is a lightweight messaging protocol that works over networks like Wi-Fi or Ethernet and is commonly used for IoT applications.

For a smart factory with **500 sensors spread over 2 km**, I would choose **LoRa with MQTT**. LoRa would collect data from all sensors over the long distance, and a central gateway would use MQTT to send the data to the cloud. This combination provides long range, low power consumption, and reliable communication.

Q3. A temperature sensor is showing impossible values. How would you find the problem?

I would troubleshoot the system step by step.

1. First, I would check whether the sensor is damaged by replacing it with another working sensor.

2. Then, I would inspect all wiring connections to make sure there are no loose or broken wires.
3. Next, I would measure the power supply using a multimeter to ensure the sensor is receiving the correct voltage.
4. If the sensor is analog, I would check whether the ADC pin of the controller is working properly.
5. Finally, I would verify the software by checking the sensor library, pin numbers, and conversion formulas.

By testing each part one by one, I can identify the exact cause of the problem.

Q4. How would you make an ESP32 weather station run for six months on battery?

To increase battery life, I would make the following changes:

- Use **Deep Sleep Mode** so the ESP32 remains asleep most of the time and wakes up only when needed.
- Read the sensors only at fixed intervals instead of continuously.
- Turn off Wi-Fi after sending the data to reduce power consumption.
- Use a low-power sensor whenever possible.
- Use a larger rechargeable battery along with a solar panel if outdoor installation is possible.
- Optimize the program by avoiding unnecessary delays and calculations.

These changes will greatly reduce power usage and help the weather station operate for several months on a single battery.

Q5. An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

For an automatic braking system, I would choose **CAN (Controller Area Network) Bus** communication. It is specially designed for vehicles and provides very fast and reliable communication with very low delay. It can also detect errors and ensure that important messages, like braking commands, are delivered immediately. Since braking is a safety-critical function, CAN Bus is the best choice instead of Wi-Fi or Bluetooth.

Q6. You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why?

I would place the sensor **at the center of the classroom**, about **1.5 to 2 meters above the floor**. This position represents the average temperature experienced by students and keeps the sensor away from direct sunlight, fans, air conditioners, windows, and doors.

If the sensor is placed near a window, it may show a higher temperature because of sunlight. If it is near an air conditioner or fan, it may show a lower temperature than the actual room temperature. Placing it near the ceiling can also give higher readings because hot air rises. Therefore, proper placement is very important for accurate measurements.

Q7. Under what situations could an analog sensor perform better than a digital one?

An analog sensor performs better when **continuous and precise measurements** are required. It provides a range of values instead of only ON or OFF.

For example:

- Measuring soil moisture accurately.
- Detecting gradual changes in light intensity using an LDR.
- Measuring temperature with small variations.
- Monitoring gas concentration levels.

Digital sensors are better for simple detection, such as whether an object is present or not, but analog sensors are more useful when detailed information is needed.

Q8. Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Block Diagram (Laser Tag Project)

Laser Gun



LDR Sensor (Player Vest)



ESP32



Health & Score Processing



ESP-NOW Communication



Opponent ESP32 / Hub ESP32



OLED Display (Health & Score)



User (Player)

Possible Failures and Solutions

Failure Point	Solution
Laser does not hit the LDR	Align the laser properly and calibrate the LDR sensitivity.
LDR gives false readings due to room light	Set a proper threshold and cover the LDR with a small tube to reduce ambient light.
ESP32 hangs or restarts	Use a stable power supply and enable the watchdog timer.
ESP-NOW communication fails	Reconnect automatically and check the MAC addresses of both ESP32 boards.
OLED display is blank	Check I2C wiring, power supply, and OLED address.
Battery becomes low	Monitor battery voltage and recharge or replace the battery when needed.
Loose wiring	Secure all connections and solder important joints.

Explanation

In my Laser Tag project, the laser beam fired by one player falls on the LDR sensor of the opponent. The ESP32 detects the hit, reduces the player's health, and sends the updated information using ESP-NOW. The health and score are then displayed on the OLED display, allowing both players to see the game status in real time. Proper calibration of the LDR, reliable wireless communication, and stable power are essential for the smooth working of the system.

Q9. If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system.

If an attacker gains access to my IoT device through the Wi-Fi network, they could perform several harmful actions. They may control the ESP32 without permission, steal sensor data, change the program settings, stop the device from working, or send fake commands to the system. In my Laser Tag project, an attacker could even manipulate the health or score of the players.

Methods to Improve Security

1. **Use a strong Wi-Fi password** so that unauthorized users cannot connect to the network.
2. **Enable WPA2/WPA3 encryption** to protect wireless communication from hackers.
3. **Use secure authentication** so that only authorized users or devices can access the ESP32.
4. **Encrypt the data** being transmitted so that even if someone captures it, they cannot understand it.
5. **Keep the ESP32 firmware updated** to fix security bugs and protect against newly discovered attacks.
6. **Disable unnecessary services and ports** to reduce the chances of attackers finding a way into the device.

By following these security measures, the IoT system becomes much safer and more reliable.

Q10. Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

When multiple devices are connected to the ESP32, some conflicts may occur.

- **GPIO pin conflict:** Two devices may be assigned the same pin. This can be solved by assigning different GPIO pins to each device.

- **Power supply issue:** If all devices draw power together, the ESP32 may restart. A separate regulated power supply should be used for high-power devices like the relay.
- **Timing conflict:** The DHT11 sensor requires accurate timing, while the ultrasonic sensor also depends on timing. Non-blocking programming or FreeRTOS tasks should be used instead of long delays.
- **I2C communication issue:** The OLED uses the I2C bus. If another I2C device is added, each device should have a unique I2C address.
- **Software delays:** Using long delay() functions can stop other devices from working properly. This can be solved using millis() or multitasking so that all sensors work smoothly.

These solutions help the ESP32 handle multiple devices efficiently without affecting performance.

Q11. An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

If the internet connection is lost, the ESP32 should **not discard the sensor readings**. Instead, it should save all the data in its local memory (such as an SD card, EEPROM, or flash memory) along with the date and time.

The ESP32 should keep checking whether the internet connection has been restored. Once the connection is available again, it should automatically upload all the stored data to the cloud in the correct order. After successful uploading, the local copy can be deleted to free up memory.

This method ensures that no important sensor data is permanently lost, even if the internet is unavailable for several hours.

Q12. Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

To make the IoT device work reliably for five years, I would improve both the hardware and software.

Hardware Improvements

- Use high-quality sensors and electronic components.
- Provide a stable and regulated power supply with surge protection.
- Use a weatherproof enclosure to protect the device from dust, moisture, and heat.
- Secure all wiring and solder the connections properly to avoid loose contacts.
- Use a rechargeable battery with solar charging if the device is installed outdoors.

Software Improvements

- Use a **watchdog timer** so that the ESP32 restarts automatically if it hangs.
- Store important settings in non-volatile memory so they are not lost after a restart.
- Add error detection and recovery mechanisms for sensors and communication.
- Enable **OTA (Over-the-Air) updates** so the software can be updated without physically accessing the device.
- Write optimized code that avoids memory leaks and unnecessary delays.

These changes will increase the reliability of the IoT device and reduce the need for maintenance.

Q13. Can an IoT system be considered "smart" if it does not use Artificial Intelligence? What differentiates “smart” from “intelligent”?

Yes, an IoT system can be considered **smart** even if it does not use Artificial Intelligence.

A smart system can collect data, communicate with other devices, and automatically perform actions based on predefined rules. For example, a smart irrigation system can automatically turn on the water pump when the soil moisture becomes low. This system is smart because it makes automatic decisions using fixed conditions.

An **intelligent** system, however, uses Artificial Intelligence or Machine Learning to learn from data and improve its decisions over time. For example, an AI-based irrigation system can study weather forecasts, soil conditions, and previous watering patterns to decide the best time to water crops.

Difference:

- **Smart system:** Works using predefined rules and automation.

- **Intelligent system:** Learns from data, adapts to changing conditions, and makes better decisions over time.

Therefore, every intelligent system is smart, but not every smart system is intelligent.

Q14. Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Polling is a method in which the microcontroller continuously checks whether a sensor or device needs attention. It is simple to implement but wastes CPU time because the controller keeps checking even when there is no new data.

Interrupt-based programming allows the microcontroller to perform other tasks and respond only when an event occurs. When the event happens, an interrupt signal immediately alerts the controller, and it executes the required task.

Polling

Interrupt

Continuously checks devices

Responds only when an event occurs

Wastes CPU time

Saves CPU time

Slower response

Faster response

Suitable for simple applications Suitable for real-time applications

Interrupts are very useful in IoT devices where a quick response is required. For example, in my **Laser Tag project**, an interrupt can immediately detect when the laser hits the LDR sensor. This ensures that the player's health is reduced instantly without waiting for the ESP32 to check the sensor repeatedly.

Q15. Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect to improve another.

In my **Laser Tag IoT project**, I made the following design trade-offs:

1. Cost vs Accuracy

I used an **LDR sensor** to detect the laser because it is inexpensive and easy to use. Although a photodiode would provide better accuracy, the LDR was sufficient for my project and kept the overall cost low.

2. Simplicity vs Features

I used **ESP-NOW** for communication instead of Wi-Fi with a cloud server. ESP-NOW is simple, fast, and works without the internet, but it does not provide remote monitoring like cloud-based systems.

3. Power Consumption vs Brightness

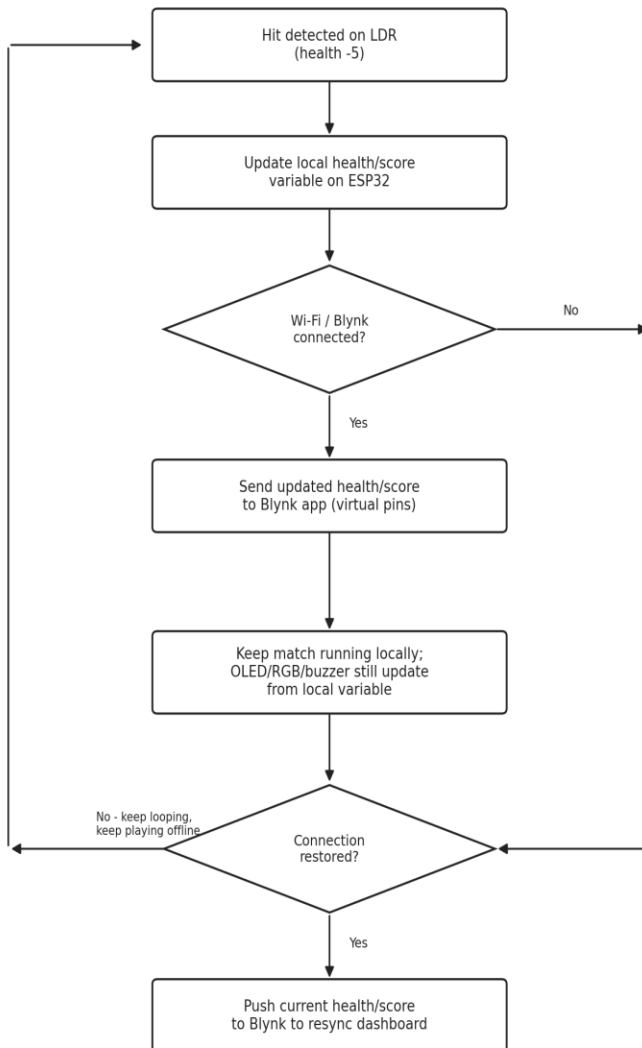
The OLED display stays ON during the game so players can continuously see their health and score. This increases power consumption, but it improves the gaming experience.

I believe these trade-offs were justified because they helped me build a low-cost, fast, and reliable prototype.

Q16. Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored.

Flowchart

Flowchart - Keep Game State When Blynk/Wi-Fi Drops



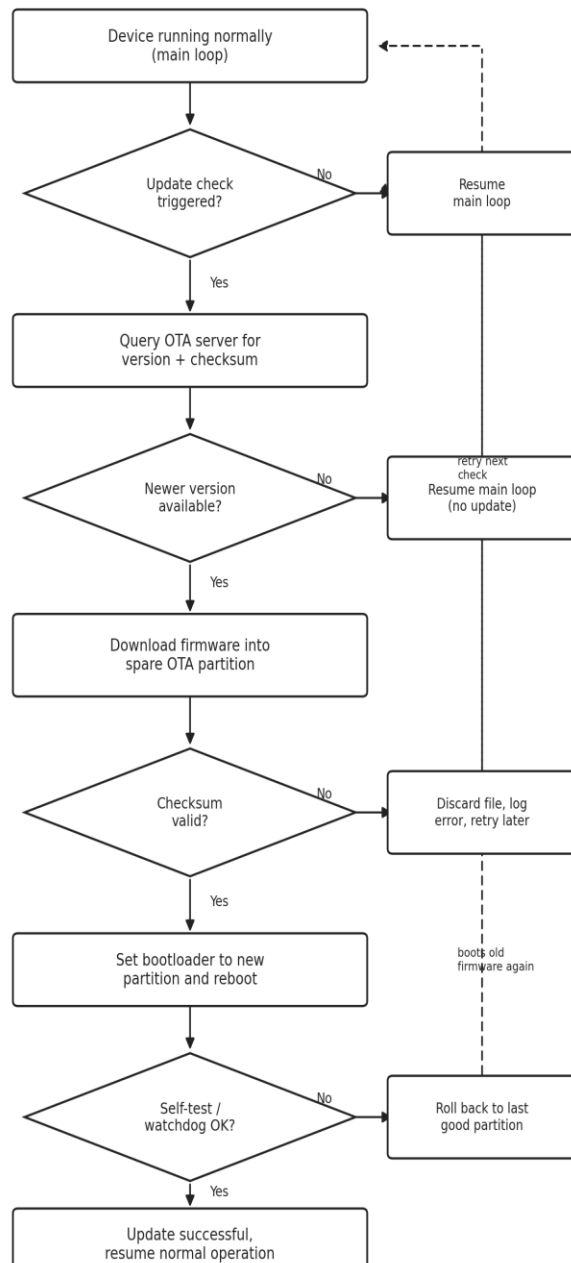
Programs/Protocols/I-O Used

- ESP32
- Sensors
- Wi-Fi
- MQTT or HTTP
- Flash Memory/SD Card
- Cloud Server (Blynk/Firebase)
- Mobile App or Web Dashboard

Q17. Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Flowchart

Flowchart - ESP32 OTA Update with Error Handling



If the update fails due to a network error or a corrupted file, the ESP32 should **keep the old firmware** and continue normal operation. This prevents the device from becoming unusable and ensures reliable operation.